MOCK EXAM 2 · ชุดที่ 3–4 · พร้อมเฉลยละเอียดท้ายเล่ม

ข้อสอบเสมือนจริง ชุดที่ 3 และ 4

ทั้งสองชุดตามเกณฑ์จริง: Choice 40 ข้อ + ตอบสั้น 10 ข้อ = 60 คะแนน · Digital Logic 15 / Computer Architecture 45

ชุดที่ 3	90 นาที · 60 คะแนน · ครอบคลุม Digital Logic และ Ch1–9
ชุดที่ 4	90 นาที · 60 คะแนน · ครอบคลุม Digital Logic และ Ch1–9 (โจทย์ไม่ซ้ำกับชุดที่ 3)
เฉลย	อยู่ท้ายเล่ม: เฉลยชุดที่ 3 ตามด้วยเฉลยชุดที่ 4 · อธิบายทุกข้อ บอกตัวลวง และอ้างกลับไปที่บทใน Exam Notes

วิธีใช้: จับเวลาจริง 90 นาทีต่อชุด ห้ามดูโน้ต ใช้เครื่องคิดเลขได้ ผิดไม่ติดลบ จึงควรตอบทุกข้อ ทำเสร็จแล้วค่อยเปิดเฉลย
ข้อตกลงทั้งเล่ม: byte addressing · 1 word = 4 bytes · 1 KB = 2^{10} B · nanoLADA ตามสไลด์ (LW 5 / ORI, ADD, SW 4 / BEQ, JMP 3 cycles)
โจทย์แต่งขึ้นตามแนวของสไลด์และ Exercise ท้ายบท ไม่ใช่ข้อสอบเก่า

ชุดที่ 3 (60 คะแนน, 90 นาที)

ส่วน	จำนวน	คะแนน	Digital Logic	Computer Architecture
Part 1: Choice	40 ข้อ × 1	40	ข้อ 1–11	ข้อ 12–40
Part 2: ตอบสั้น	10 ข้อ × 2	20	S1–S2	S3–S10
รวม		**60**	**15**	**45**

Part 1: Choice (ข้อละ 1 คะแนน)

Digital Logic (ข้อ 1–11)

1. $(2B)_{16}$ มีค่าเท่าใดในฐานสิบ

a) 32 b) 43
c) 53 d) 211

2. 25.625 ในฐานสิบ เขียนเป็นฐานสองได้ว่า

a) 11001.101 b) 11001.011
c) 10011.101 d) 11010.101

3. 1111 0110 ในระบบ 8-bit two's complement มีค่าเท่าใด

a) –9 b) 246
c) –10 d) –118

4. Sign-extend เลข 4-bit two's complement `1010` ให้เป็น 8 bits ได้ผลเป็น

a) 0000 1010 b) 1000 1010
c) 1111 0101 d) 1111 1010

5. การบวกเลข 8-bit two's complement คู่ใดเกิด **signed overflow**

a) 0x7F + 0x01 b) 0x80 + 0x7F
c) 0xFF + 0x01 d) 0x40 + 0xC0

6. A + A'B ลดรูปได้เป็น

a) A b) A + B
c) AB d) A'B

7. F(A,B,C) = Σm(1, 3, 5, 7) ลดรูปได้เป็น

a) A b) B'
c) C d) A + C

8. ฟังก์ชัน 3 ตัวแปร F = Σm(0, 1, 2) เขียนในรูป POS ได้เป็น

a) ΠM(0, 1, 2)
b) ΠM(3, 4, 5, 6, 7)
c) ΠM(5, 6, 7)
d) Σm(3, 4, 5, 6, 7)

9. Decoder 3-to-8 ได้รับอินพุต ABC = 101 output เส้นใดเป็น 1

a) Y2
b) Y3
c) Y5
d) Y6

10. T flip-flop มี Q = 1 และ T = 1 หลังขอบ clock ค่า Q คือ

a) 0
b) 1
c) คงเดิม
d) ไม่แน่นอน (forbidden)

11. FSM ที่มี 12 states ถ้าใช้ **one-hot encoding** ต้องใช้ flip-flop กี่ตัว

a) 4
b) 6
c) 12
d) 24

Computer Architecture (ข้อ 12–40)

12. ข้อใด**ไม่ใช่**ส่วนประกอบของ Computer Organization ตามสไลด์

a) ALU
b) Bus
c) Control Unit
d) Instruction format

13. การที่ซอฟต์แวร์ของ 8088 ยังรันบน Core i7 ได้ แสดงถึงข้อใด

a) Moore's Law
b) ISA เป็น abstraction of software for hardware (backward compatibility)
c) Core i7 มี organization เหมือน 8088
d) Amdahl's Law

14. โปรแกรมเดิมใช้เวลา 10 s หลังปรับปรุงเหลือ 4 s ได้ speedup เท่าใด

a) 0.4
b) 1.5
c) 2.5
d) 6

15. การใช้ quicksort หรือ merge sort สั้น ๆ เป็นตัววัด performance จัดเป็น benchmark ประเภทใด

a) Real
b) Kernel
c) Synthetic
d) Toy

16. CPU 500 MHz รันโปรแกรม 2×10^8 คำสั่ง ด้วย CPI เฉลี่ย 2.5 จะใช้ CPU time เท่าใด

a) 0.1 s
b) 0.25 s
c) 1 s
d) 2.5 s

17. เครื่องในข้อ 16 มีค่า MIPS เท่าใด

a) 80
b) 200
c) 500
d) 1250

18. โปรแกรมที่ทำ parallel ได้ 75% ของเวลา จะได้ speedup **สูงสุด**เท่าใด (cores → ∞)

a) 1.33
b) 3
c) 4
d) ไม่มีขีดจำกัด

19. ถ้ารู้สัดส่วนการใช้งานจริงของแต่ละโปรแกรม และต้องการรวม**เวลา**ที่วัดได้เป็นค่าเดียว ควรใช้ค่าเฉลี่ยแบบใด

a) Weighted arithmetic mean
b) Geometric mean
c) Harmonic mean
d) Median

20. สถาปัตยกรรมแบบใดคำนวณ x = (a·b) + (a – (b·c)) ได้ด้วย**คำสั่งน้อยที่สุด (4 คำสั่ง)** ตามสไลด์

a) Memory-Memory (3 operands)
b) Load-Store
c) Stack
d) Accumulator

21. เก็บค่า 0xDEADBEEF แบบ **little endian** ที่ address 0x40 ไบต์ที่ address 0x40 คือ

a) 0xDE
b) 0xAD
c) 0xBE
d) 0xEF

22. โค้ด C `A = **p;` ตรงกับ addressing mode ใด

a) Register indirect
b) Memory indirect
c) Displacement
d) Auto-increment

23. ข้อใดถูกต้องเกี่ยวกับ **unrestricted alignment**

a) compiler ต้องใส่ padding ให้ทุกตัวแปร
b) hardware จะ trap ทุกครั้งที่ข้อมูลไม่ aligned
c) hardware ต้องตรวจและแก้เอง ซึ่งอาจต้องเข้าถึง memory 2 ครั้งขึ้นไป
d) เร็วกว่า restricted alignment เสมอ

24. Exception ที่เกิดจาก**ซอฟต์แวร์** เช่น system call หรือการหารด้วยศูนย์ เรียกว่า

a) Interrupt
b) Trap
c) Vector
d) Bubble

25. การหา handler แบบ **Fixed Entry** ทำงานอย่างไร

a) อ่าน address ของ handler จากตารางตามหมายเลข interrupt
b) กระโดดไปที่ Base + หมายเลข × ขนาด handler
c) กระโดดไป address คงที่เสมอ แล้วให้ software อ่าน interrupt-source register เพื่อตัดสินใจ
d) hardware เลือก handler ตาม priority เอง

26. ใน nanoLADA single-cycle ค่า control ของคำสั่ง **LW** ข้อใดถูก

a) sel_b = 1, sel_data = 1, reg_wr = 1
b) sel_b = 0, sel_data = 0, reg_wr = 1
c) sel_b = 1, sel_data = x, mem_wr = 1
d) sel_pc = 1, reg_wr = 0

27. คำสั่ง nanoLADA ใดใช้ extender แบบ **zero extend**

a) ORI
b) ORUI
c) LW
d) BEQ

28. คำสั่ง `JMP` ใน nanoLADA มี address field = 0x40 ค่า PC ถัดไปคือ

a) 0x40
b) 0x44
c) 0x100
d) 0x104

29. ใน multi-cycle nanoLADA cycle ที่ 4 ของคำสั่ง **SW** ทำอะไร

a) IR ← MEM[PC]
b) REG_S ← REG_A + REG_B
c) MEM[REG_S] ← REG_M
d) R[rt] ← REG_WR

30. ซักผ้าแบบ pipeline 4 ขั้นที่ใช้เวลา 30, 40, 30, 20 นาที เมื่อ pipeline เต็มแล้วจะมีผ้าเสร็จ 1 load ทุกกี่นาที

a) 20
b) 30
c) 40
d) 120

31. Branch delay slot ต้องอาศัยสิ่งใดจึงจะได้ประโยชน์

a) จำนวน register มากขึ้น
b) compiler ที่หาคำสั่งอิสระมาใส่ใน slot
c) TLB
d) write buffer

32. วงจร hazard detection ของ pipeline เปรียบเทียบอะไร

a) opcode ของคำสั่งที่ติดกัน
b) rs/rt ของคำสั่งปัจจุบัน กับ register ปลายทางที่รอเขียนของคำสั่งใน EX/MEM/WB
c) ค่า PC ของทุก stage
d) address ของ memory ที่กำลังอ่าน

33. เทคนิคใด**กำจัด WAR และ WAW** ที่เกิดจาก out-of-order execution ได้

a) Forwarding
b) Branch prediction
c) Register renaming
d) Write buffer

34. VLIW เพิ่ม performance โดย**ลดตัวแปรใด**ใน CPU time เป็นหลัก

a) IC
b) CPI
c) T_c
d) Miss rate

35. ข้อใดเป็นตัวอย่างของ SIMD

a) Intel AVX
b) Server แบบ multicore
c) Uniprocessor 8088
d) ระบบ fault-tolerant ที่ให้หลายเครื่องคำนวณแล้ว vote

36. ข้อใดเป็นตัวอย่างของ **temporal locality**

a) อ่าน array เรียงจาก index 0 ไปเรื่อย ๆ
b) ตัวแปร i ใน loop ถูกใช้ซ้ำทุกรอบ
c) คำสั่งถูก fetch เรียงตาม address
d) block size 64 bytes

37. 2-way set associative cache ขนาด 8 KB มี block size 32 bytes จะมีกี่ sets

a) 64
b) 128
c) 256
d) 512

38. Cache แบบ write-back ต้องมีบิตใดเพิ่มในแต่ละ line

a) Parity bit
b) Dirty bit
c) Sign bit
d) Carry bit

39. Page fault ถูกจัดการโดยใคร

a) Cache controller
b) TLB
c) Operating system (software)
d) Compiler

40. NAS ใช้ protocol ใดในการแชร์ไฟล์

a) iSCSI
b) Fibre Channel
c) NFS
d) SATA

Part 2: ตอบสั้น (ข้อละ 2 คะแนน)

Digital Logic

S1. ต้องใช้ 2-input **NAND gate อย่างน้อยกี่ตัว**จึงจะสร้างฟังก์ชัน A + B ได้ และเขียนนิพจน์ที่ใช้

S2. ลดรูป F(A,B,C,D) = Σm(0, 2, 5, 7, 8, 10, 13, 15) ให้เป็น SOP ที่สั้นที่สุด

Computer Architecture

S3. เครื่อง A และ B ใช้ ISA เดียวกัน (IC เท่ากัน) โดย A: 2 GHz, CPI 2.0 และ B: 3 GHz, CPI 2.7 เครื่องใดเร็วกว่า และเร็วกว่ากี่เท่า

S4. งาน floating-point กินเวลา 40% ของโปรแกรม ถ้าทำให้ FP เร็วขึ้น 5 เท่า overall speedup เป็นเท่าใด

S5. ระบบมี interrupt หมายเลข 6

- (ก) ถ้าใช้ **Vector Table** ที่ base 0x2000 และ address 32 bits CPU จะอ่าน address ของ handler จาก address ใด
- (ข) ถ้าใช้ **Handler Table** ที่ base 0x2000 และ handler ขนาด 64 bytes CPU จะกระโดดไปที่ address ใด

S6. ใช้ delay ของ single-cycle nanoLADA ตามสไลด์ (PC 10, Inst MEM 40, Control 40, Register 20, MUX 10, ALU 60, Data MEM 40, MUX 10 ns) คำสั่ง **ADD** ใช้เวลาจริงเท่าใด และ clock cycle ของเครื่องต้องตั้งเป็นเท่าใด

S7. 5-stage pipeline (full forwarding) cycle time 2 ns รัน 200 คำสั่ง โดย 10% ของคำสั่งเป็น load ที่มีคำสั่งถัดไปใช้ค่าทันที (load-use) และไม่มี stall อื่น จะใช้เวลาเท่าใด

S8. Direct-mapped cache ขนาด 4 KB, block 16 bytes, address 32 bits จงหา **index และ tag** (ฐานสิบ) ของ address 0x00001234

S9. CPU มี CPI_{ideal} = 1.2 และ 25% ของคำสั่งเป็น load/store โดย I-cache miss rate 1%, D-cache miss rate 4% และ miss penalty 80 cycles จงหา CPI จริง

S10. Virtual address 32 bits, page size 16 KB และ PTE ขนาด 4 bytes จงหาจำนวนบิตของ **VPN** และ**ขนาดของ page table** ต่อ process

---

ชุดที่ 4 (60 คะแนน, 90 นาที)

ส่วน	จำนวน	คะแนน	Digital Logic	Computer Architecture
Part 1: Choice	40 ข้อ × 1	40	ข้อ 1–11	ข้อ 12–40
Part 2: ตอบสั้น	10 ข้อ × 2	20	S1–S2	S3–S10
รวม		**60**	**15**	**45**

Part 1: Choice (ข้อละ 1 คะแนน)

Digital Logic (ข้อ 1–11)

1. $(1101\ 0110)_2$ เขียนเป็นฐานสิบหกได้ว่า

a) 0xD6
b) 0x6D
c) 0xC6
d) 0xD5

2. 345_8 มีค่าเท่าใดในฐานสิบ

a) 345
b) 229
c) 237
d) 197

3. เลข **unsigned 10 bits** มีช่วงค่าเท่าใด

a) 0 ถึง 1023
b) –512 ถึง 511
c) 0 ถึง 1024
d) 0 ถึง 511

4. –90 ในระบบ 8-bit **one's complement** คือ

a) 1010 0110
b) 1010 0101
c) 1101 1010
d) 0101 1010

5. บวกเลข 8-bit two's complement 0x90 + 0xA0 ข้อใดถูก

a) carry out = 0, overflow = 0
b) carry out = 1, overflow = 0
c) carry out = 0, overflow = 1
d) carry out = 1, overflow = 1

6. A $\oplus$ B เท่ากับข้อใด

a) AB + A'B'
b) A'B + AB'
c) (A + B)'
d) A'B'

7. AB + A'C + BC ลดรูปได้เป็น

a) AB + A'C
b) AB + BC
c) A + C
d) BC

8. F(A,B,C) = Σm(0, 2, 4, 6) ลดรูปได้เป็น

a) A'
b) C'
c) B'
d) A'C'

9. ต้องใช้ MUX แบบ 2-to-1 กี่ตัวจึงจะสร้าง MUX แบบ 8-to-1 ได้

a) 3
b) 4
c) 7
d) 8

10. อุปกรณ์ใดเปลี่ยนค่าเฉพาะ**ที่ขอบ (edge)** ของ clock

a) SR latch
b) D latch
c) D flip-flop
d) Combinational circuit

11. Mealy machine ที่ตรวจจับลำดับ "101" แบบ overlapping ต้องใช้อย่างน้อยกี่ state

a) 2
b) 3
c) 4
d) 5

Computer Architecture (ข้อ 12–40)

12. ในลำดับชั้นของ abstraction ชั้นใดอยู่ระหว่าง compiler/OS กับ hardware (datapath & control)

a) ABI
b) ISA
c) Datapath
d) Layout

13. Pipelining ปรับปรุงสิ่งใดเป็นหลัก

a) Latency ของแต่ละคำสั่ง
b) Instruction count
c) Throughput
d) Miss penalty

14. ในผลลัพธ์ของคำสั่ง `time` บน Unix ค่า **real** คือเวลาใด

a) CPU time ใน user mode
b) CPU time ใน kernel
c) Elapsed (wall-clock) time
d) CPU time ของทุก process รวมกัน

15. X ใช้เวลา 30 s และ Y ใช้เวลา 20 s กับโปรแกรมเดียวกัน Y เร็วกว่า X กี่เท่า

a) 0.67
b) 1.5
c) 10
d) 50

16. โปรแกรมหนึ่งใช้ 3×10^9 clock cycles บน CPU 1.5 GHz จะใช้ CPU time เท่าใด

a) 0.5 s
b) 3 s
c) 2 s
d) 4.5 s

17. Instruction mix: 60% มี CPI 1, 30% มี CPI 2 และ 10% มี CPI 5 average CPI คือ

a) 1.7
b) 2.67
c) 8
d) 1.5

18. ทำให้ส่วนที่กินเวลา 50% เร็วขึ้น 2 เท่า จะได้ overall speedup เท่าใด

a) 1.33
b) 1.5
c) 2
d) 1.25

19. Speedup ของสองโปรแกรมคือ 2 และ 8 geometric mean คือ

a) 4
b) 5
c) 16
d) 3.2

20. ข้อใดเป็นข้อเสียของ **Accumulator architecture** ตามสไลด์

a) ต้องใช้ register จำนวนมาก
b) decode ซับซ้อน
c) code density แย่ที่สุด
d) memory traffic สูง และ accumulator เป็น bottleneck

21. ข้อใดอธิบายยุค **Post-RISC** ได้ถูกต้อง

a) x86 แปลคำสั่ง CISC เป็นคำสั่งภายในคล้าย RISC (micro-ops) ก่อนทำ parallelism
b) RISC ถูกเลิกใช้ไปแล้ว
c) ARM กลายเป็น CISC
d) x86 เปลี่ยนเป็น fixed-length instruction

22. ภายใต้ restricted alignment ถ้าประกาศ `char a; double b;` เริ่มที่ address 0 ตัวแปร b จะเริ่มที่ address ใด

a) 1
b) 2
c) 4
d) 8

23. compiler เปลี่ยน `x * 8` เป็น `x << 3` คือ optimization แบบใด

a) Code motion
b) Strength reduction
c) Common subexpression elimination
d) Procedure inlining

24. ข้อดีหลักของ branch แบบ **PC-relative** คือ

a) กระโดดไป address ใดก็ได้ใน memory
b) ไม่ต้องใช้ adder
c) โปรแกรม relocatable (ย้ายไปวางที่ใดก็ทำงานได้)
d) ทำให้ memory เร็วขึ้น

25. Callee-save หมายถึงข้อใด

a) ผู้เรียก PUSH ทุก register ก่อน CALL
b) ไม่มีใคร save register
c) OS เป็นผู้ save register
d) subroutine save เฉพาะ register ที่ตัวเองจะแก้ แล้ว restore ก่อน RET

26. ใน nanoLADA single-cycle ค่า control ของคำสั่ง **BEQ** ข้อใดถูก

a) alu_ops = 10 (SUB), sel_addpc = zero, reg_wr = 0
b) alu_ops = 00, reg_wr = 1
c) sel_pc = 1, mem_wr = 1
d) sel_data = 1, reg_wr = 1

27. Register file ของ nanoLADA มีกี่ port

a) อ่าน 1, เขียน 1
b) อ่าน 2, เขียน 2
c) อ่าน 2, เขียน 1
d) อ่าน 3, เขียน 1

28. multi-cycle nanoLADA (cycle time 25 ns) รัน ADD 4 คำสั่ง, SW 2 คำสั่ง, BEQ 2 คำสั่ง และ JMP 1 คำสั่ง จะใช้เวลาเท่าใด

a) 225 ns
b) 825 ns
c) 900 ns
d) 1125 ns

29. ทำไมจึงไม่ให้ programmer เขียนโปรแกรมด้วย microinstruction โดยตรง

a) microinstruction ทำงานช้ากว่า
b) microinstruction ไม่ Turing complete
c) ต้องใช้ OS พิเศษ
d) microinstruction ผูกกับ organization ของแต่ละรุ่น จึงไม่ compatible ข้ามรุ่น

30. ถ้ามี memory ตัวเดียวทั้งคำสั่งและข้อมูล pipeline จะเกิด structural hazard วิธีแก้ที่สไลด์เสนอคือ

a) Forwarding
b) แทรก NOP เสมอ
c) แยก instruction cache กับ data cache
d) Branch prediction

31. โค้ด `a = 7; b = a + 5;` เป็น dependency ชนิดใด

a) RAW
b) WAR
c) WAW
d) Structural

32. 5-stage pipeline ที่มี full forwarding ถ้า LW ตามด้วยคำสั่งที่ใช้ค่านั้นทันที ต้อง stall กี่ cycle

a) 0
b) 1
c) 2
d) 3

33. ตามสไลด์ branch prediction แบบใช้ branch history ทายถูกประมาณเท่าใด

a) 50%
b) 75%
c) 90%
d) 100%

34. Superscalar degree 4 กับ pipeline 5 stages รัน 1000 คำสั่ง (ideal) ใช้เวลากี่ T_c

a) 254
b) 1004
c) 250
d) 259

35. ข้อดีของ **dynamic scheduling** (superscalar OoO) เทียบกับ static scheduling คือ

a) hardware ง่ายกว่า
b) compiler เห็น scope กว้างกว่า
c) ใช้ power ต่ำกว่า
d) software เดิมเร็วขึ้นบน hardware ใหม่โดยไม่ต้อง recompile

36. "Ping-pong effect" (cache มี 1 line ทุก address ใหม่ miss หมด) เป็นกรณีสุดโต่งของ miss ชนิดใด

a) Compulsory
b) Capacity
c) Conflict
d) Coherence

37. ถ้า cache size คงที่แต่เพิ่ม block size มากเกินไป จะเกิดอะไรขึ้น

a) จำนวน block น้อยลง temporal locality แย่ลง miss rate กลับเพิ่ม และ miss penalty สูงขึ้น
b) miss rate ลดลงเสมอ
c) hit time เป็นศูนย์
d) ไม่มี compulsory miss อีกต่อไป

38. page size 4 KB และต้องการ L1 cache ขนาด 16 KB ที่ทำ overlapped cache & TLB access ได้ ต้องใช้ associativity อย่างน้อยเท่าใด

a) 2-way
b) 4-way
c) 8-way
d) 16-way

39. ทำไม virtual memory จึงใช้ write-back เท่านั้น

a) write-through ไปยังดิสก์ช้าและแพงเกินไป
b) ดิสก์เขียนข้อมูลไม่ได้
c) OS ติดตาม dirty page ไม่ได้
d) TLB บังคับไว้

40. RAID 6 ที่ใช้ดิสก์ 4 TB จำนวน 6 ลูก มีพื้นที่ใช้งานจริงเท่าใด

a) 24 TB
b) 20 TB
c) 16 TB
d) 12 TB

Part 2: ตอบสั้น (ข้อละ 2 คะแนน)

Digital Logic

S1. ถ้าต้องการให้ flip-flop เปลี่ยนจาก Q = 0 เป็น $Q^+ = 1$ ต้องป้อนอินพุตอะไรให้ (ก) **JK flip-flop** (ค่า J, K) และ (ข) **T flip-flop**

S2. เขียน –37 ในระบบ 8-bit two's complement เป็นเลขฐานสองและฐานสิบหก

Computer Architecture

S3. โปรแกรมมี 6×10^9 คำสั่ง CPI 1.25 รันบน CPU 2.5 GHz จงหา **CPU time** และ **MIPS**

S4. มีโปรแกรม 2 ตัว ใช้งานจริงในสัดส่วน P1 70% และ P2 30% เวลาที่วัดได้ (วินาที) คือ Machine A: P1 = 8, P2 = 20 และ Machine B: P1 = 12, P2 = 6 จงหา **weighted arithmetic mean** ของแต่ละเครื่อง และบอกว่าควรเลือกเครื่องใด

S5. เก็บค่า 0x0A0B0C0D ที่ address 0x100 แบบ **little endian** จงเขียนค่าของ byte ที่ address 0x100, 0x101, 0x102, 0x103 และบอกว่าถ้าเป็น **big endian** byte ที่ 0x102 คืออะไร

S6. Single-cycle มี cycle time 250 ns ส่วน multi-cycle nanoLADA มี cycle time 60 ns โดย instruction mix คือ LW 25%, SW 10%, R-type/ORI 45%, BEQ 15%, JMP 5% จงหา **average CPI** ของ multi-cycle และบอกว่าเครื่องใดเร็วกว่า

S7. พิจารณาโค้ดบน 5-stage pipeline ที่มี full forwarding ระบุ dependency ทั้งหมด (ชนิดและ register) และอธิบายว่า **ทำไมย้าย SUB มาไว้ระหว่างคำสั่ง 1 กับ 2 เพื่อลด stall ไม่ได้** และต้องทำอย่างไรจึงจะย้ายได้

```
1: LW  $r2, 0($r1)
2: ADD $r3, $r2, $r4
3: SUB $r2, $r5, $r6
4: SW  $r3, 4($r1)
```

S8. 2-way set associative cache ขนาด 32 KB, block 32 bytes, address 32 bits จงหาจำนวนบิตของ **tag, index, offset** และจำนวน comparator

---

S9. Cache เดิมมี hit time 2 cycles, miss rate 8%, miss penalty 50 cycles ถ้าเปลี่ยนเป็น 2-way ทำให้ miss rate ลดเหลือ 6% แต่ hit time เพิ่มเป็น 2.5 cycles จงหา AMAT ของทั้งสองแบบ และบอกว่าแบบใดดีกว่า

S10. มีดิสก์ 3 TB จำนวน 5 ลูก จงหาพื้นที่ใช้งานจริงและจำนวนดิสก์ที่เสียได้โดยข้อมูลไม่หาย สำหรับ **RAID 0, RAID 5 และ RAID 6**

เฉลยละเอียด ชุดที่ 3

ตารางคำตอบ (ตรวจเร็ว)

ข้อ	1	2	3	4	5	6	7	8	9	10
ตอบ	b	a	c	d	a	b	c	b	c	a
ข้อ	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**
ตอบ	c	d	b	c	d	c	b	c	a	a
ข้อ	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**
ตอบ	d	b	c	b	c	a	a	c	c	c
ข้อ	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**	**40**
ตอบ	b	b	c	a	a	b	b	b	c	c

แยกคะแนน: Digital Logic = ข้อ 1–11 + S1–S2 (เต็ม 15) · Computer Architecture = ข้อ 12–40 + S3–S10 (เต็ม 45)

ข้อไหนอยู่บทไหน (ใช้หาว่าควรกลับไปอ่านส่วนไหนใน Exam Notes):

บท	ข้อ
Digital Logic	1–11, S1, S2
Ch1 Introduction	12, 13
Ch2 Performance	14–19, S3, S4
Ch3 ISA	20–25, S5
Ch4 Single cycle	26–28, S6
Ch5 Multi cycle	29
Ch6 Pipelining	30–33, S7
Ch7 ILP	34, 35
Ch8 Memory	36–39, S8, S9, S10
Ch9 Storage	40

Part 1: Choice

Digital Logic

1. ตอบ b) 43

$(2B)_{16} = 2 \times 16 + 11 =$ **43**

- **ตัวลวง a:** ลืมแปลง B (= 11)
- **ตัวลวง d (211):** 0x2B = 0010 1011 อ่านผิดเป็นเลขฐานสิบตรง ๆ

2. ตอบ a) 11001.101

- ส่วนจำนวนเต็ม: 25 = 16 + 8 + 1 = **11001**
- ส่วนทศนิยม (คูณ 2 ซ้ำ): 0.625 × 2 = **1**.25 → 0.25 × 2 = **0**.5 → 0.5 × 2 = **1**.0 → อ่านจากบนลงล่างได้ **.101**
- **ตัวลวง b (.011):** อ่านหลักทศนิยมกลับด้าน

3. ตอบ c) –10

MSB = 1 จึงเป็นค่าลบ ให้กลับบิตแล้วบวก 1 เพื่อหาขนาด: 0000 1001 + 1 = 0000 1010 = 10 → **–10**

- **ตัวลวง a (–9):** อ่านแบบ 1's complement
- **ตัวลวง b (246):** อ่านแบบ unsigned
- **ตัวลวง d (–118):** อ่านแบบ sign-magnitude

4. ตอบ d) 1111 1010

Sign extension = **คัดลอก sign bit (1) ไปเติมทางซ้าย** → 1010 (–6) กลายเป็น 1111 1010 (–6) ค่าเท่าเดิม

- **ตัวลวง a:** zero extension ซึ่งค่าจะเปลี่ยนเป็น +10
- **ตัวลวง c:** กลับบิตแทนการขยาย

5. ตอบ a) 0x7F + 0x01

127 + 1 = 128 เกินค่าสูงสุด 127 → ได้ 0x80 (= –128) ซึ่งเป็น**บวก + บวก ได้ลบ** → overflow

- b) –128 + 127 = –1: ต่างเครื่องหมาย ไม่มีทาง overflow
- c) –1 + 1 = 0: มี carry out แต่**ไม่ overflow**
- d) 64 + (–64) = 0: ต่างเครื่องหมาย

6. ตอบ b) A + B

กฎ **A + A'B = A + B**

- พิสูจน์: A + A'B = (A + A')(A + B) = 1 · (A + B) ตามกฎ distributive แบบ OR
- **ตัวลวง a:** เป็นกฎ absorption ซึ่งใช้กับ A + AB (ไม่มีขีด)

7. ตอบ c) C

m1, m3, m5, m7 = 001, 011, 101, 111 → **C = 1 ทุกตัว** ส่วน A, B เปลี่ยนครบทุกค่า → F = C (เป็นแถวคอลัมน์ C = 1 ทั้งหมดใน K-map)

8. ตอบ b) ΠM(3, 4, 5, 6, 7)

POS ใช้**แถวที่ F = 0** คือแถวที่เหลือจาก Σm: {0..7} – {0, 1, 2} = {3, 4, 5, 6, 7}

- **ตัวลวง a:** ใช้เลขเดียวกับ Σm ซึ่งผิด
- **ตัวลวง d:** คือ **F'** ไม่ใช่ F

9. ตอบ c) Y5

$101_2 = 5$ decoder ทำให้ output เส้นที่ 5 เป็น 1 เพียงเส้นเดียว (one-hot) เพราะ Y5 = minterm m5 = AB'C

10. ตอบ a) 0

$Q^+ = T \oplus Q = 1 \oplus 1 =$ **0** (T = 1 คือ toggle)

11. ตอบ c) 12

One-hot ใช้ FF หนึ่งตัวต่อหนึ่ง state ดังนั้น 12 states ใช้ 12 FFs

- **ตัวลวง a (4):** เป็นจำนวนของ **binary encoding** = $\lceil \log_2 12 \rceil = 4$
- one-hot ใช้ FF มากกว่า แต่ decode state ง่ายกว่า

Computer Architecture

12. ตอบ d) Instruction format

Instruction format เป็นส่วนของ **ISA** (software มองเห็น) ส่วน organization ในสไลด์ประกอบด้วย ALU, Control Unit, Register, Bus, Memory subsystem และ I/O subsystem

13. ตอบ b)

สไลด์ Ch1 ใช้ตัวอย่างนี้อธิบาย ISA ในมุม **"abstraction of software for hardware"** คือ hardware รุ่นใหม่ต้องรักษา ISA เดิมไว้ ซอฟต์แวร์เก่าจึงยังรันได้

- **ตัวลวง c:** organization ของ Core i7 ต่างจาก 8088 มาก มีแค่ ISA ที่ compatible กัน

14. ตอบ c) 2.5

Speedup = $T_{old} / T_{new} = 10/4 =$ **2.5**

- **ตัวลวง a:** กลับเศษส่วน
- **ตัวลวง d:** ผลต่าง (10 – 4)

15. ตอบ d) Toy

Toy benchmark คือโปรแกรมเล็ก ๆ อย่าง quicksort/merge sort ที่นิยมในอดีต ส่วน synthetic คือโปรแกรมที่เขียนขึ้นให้**เลียนแบบ**พฤติกรรมของโปรแกรมจริง (Whetstone, Dhrystone) และ kernel คือการวัดเฉพาะบางแง่มุมของ performance

16. ตอบ c) 1 s

CPU time = IC × CPI / f = $(2 \times 10^8 \times 2.5) / (5 \times 10^8) = 5 \times 10^8 / 5 \times 10^8 =$ **1 s**

17. ตอบ b) 200

MIPS = f / (CPI × 10^6) = $500 \times 10^6 / (2.5 \times 10^6) =$ **200**

ตรวจด้วยอีกสูตร: IC / (T × 10^6) = $2 \times 10^8 / (1 \times 10^6) = 200$ ✓

18. ตอบ c) 4

$S_{max} = 1 / (1 - P) = 1 / 0.25 =$ **4** เพราะส่วน serial 25% จำกัด speedup (ตัวลวง d ลืม Amdahl)

19. ตอบ a) Weighted arithmetic mean

เวลาเป็นค่าที่มีหน่วย และรู้สัดส่วน workload จริง จึงใช้ weighted AM ส่วน geometric mean ใช้กับ **ratio** (speedup) ที่ไม่มีหน่วย

20. ตอบ a) Memory-Memory (3 operands)

แบบ	จำนวนคำสั่ง
Memory-Memory 3-operand	**4**
Memory-Memory 2-operand	7
Load-Store	8
Stack / Register-Memory	10
Accumulator	11

ข้อแลกเปลี่ยนคือ memory traffic สูงมาก

21. ตอบ d) 0xEF

Little endian เก็บ **byte ต่ำ (LSB) ไว้ที่ address ต่ำ**: 0x40 = EF, 0x41 = BE, 0x42 = AD, 0x43 = DE

- **ตัวลวง a:** เป็นคำตอบของ big endian

22. ตอบ b) Memory indirect

`**p` ต้องอ่าน memory 2 ครั้ง: M[M[R2]] = `LD r1, @(r2)`

- Register indirect = `*p` (อ่านครั้งเดียว)
- Auto-increment = `D[i++]`

23. ตอบ c)

Unrestricted alignment: software ง่าย แต่ **hardware ต้องตรวจและแก้ misalignment เอง** มักต้องเข้าถึง memory 2 ครั้งขึ้นไป (ช้า, logic แพง)

- a และ b เป็นลักษณะของ **restricted** alignment
- d ผิด: สไลด์สรุปว่า restricted ชนะในแง่ performance

24. ตอบ b) Trap

Exception แบ่งตามแหล่ง: **interrupt** มาจาก hardware ภายนอก ส่วน **trap** มาจาก software (Intel เรียกทั้งสองแบบว่า interrupt)

25. ตอบ c)

Fixed entry: CPU กระโดดไป address เดิมเสมอ (สไลด์ใช้ 0x1000) แล้วให้ software อ่าน interrupt-source register

- a = vector table
- b = handler table

26. ตอบ a)

LW: sel_b = 1 (ใช้ immediate คำนวณ address), sel_data = 1 (เขียนข้อมูลจาก memory), sel_wr = 1 (rt), reg_wr = 1, mem_wr = 0, alu_ops = 00 (ADD)

- **ตัวลวง b:** เป็นค่าของ ADD
- **ตัวลวง c:** เป็นค่าของ SW
- **ตัวลวง d:** เป็นค่าของ JMP

27. ตอบ a) ORI

- **ORI → zero extend** (0x0000_imm)
- **ORUI → zero pad** (imm_0000)
- **LW/SW/BEQ → sign extend**

28. ตอบ c) 0x100

RTL: PC ← addr × 4 = 0x40 × 4 = **0x100**

- **ตัวลวง a:** ลืมคูณ 4
- **ตัวลวง d:** บวก PC + 4 ซึ่ง JMP ไม่ใช้ (JMP ไม่ได้เป็น PC-relative)

29. ตอบ c) MEM[REG_S] ← REG_M

SW 4 cycles:

1. Fetch
2. REG_A ← R[rs], REG_B ← sign_ext(imm), **REG_M ← R[rt]**
3. REG_S ← REG_A + REG_B
4. **MEM[REG_S] ← REG_M**

- **ตัวลวง d:** เป็น write back ซึ่ง SW ไม่มี

30. ตอบ c) 40

ความเร็วของ pipeline ถูกจำกัดด้วย stage ที่ช้าที่สุด (Lessons learned) ทุก stage ต้องเดินตามจังหวะ 40 นาที จึงเสร็จ 1 load ทุก 40 นาที

- **ตัวลวง d (120):** เป็นผลรวมเวลาของทุกขั้นของผ้า load เดียว (latency) ไม่ใช่อัตราที่ผ้าเสร็จ

31. ตอบ b)

Delay slot = คำสั่งหลัง branch ทำงานเสมอ จึงต้องให้ **compiler/software** หาคำสั่งที่ไม่เกี่ยวกับ branch มาใส่ ถ้าหาไม่ได้ต้องใส่ NOP (สไลด์: "Need support from software")

32. ตอบ b)

สมการในสไลด์: hazard = (rs = rw_{ex} AND $regW_{ex}$) OR (rs = rw_{mem} AND $regW_{mem}$) OR (rs = rw_{wb} AND $regW_{wb}$) และเงื่อนไขเดียวกันกับ rt คือเทียบ **operand ของคำสั่งปัจจุบัน กับ pending writes**

33. ตอบ c) Register renaming

WAR/WAW เป็น **name dependence** (ใช้ชื่อ register ซ้ำ) ถ้าเปลี่ยนให้ใช้ physical register คนละตัวก็หายไป

- **ตัวลวง a:** forwarding แก้ **RAW**
- **ตัวลวง b:** branch prediction แก้ **control hazard**

34. ตอบ a) IC

VLIW รวม x operations ใน 1 คำสั่ง → IC = n/x (CPI = 1) ส่วน superscalar ลด **CPI** (1/x) และ superpipeline ลด CPI ที่มีผลด้วย minor cycle

35. ตอบ a) Intel AVX

AVX ทำ operation เดียวกับหลายข้อมูลในคำสั่งเดียว (SIMD/vector)

- b = MIMD
- c = SISD
- d = MISD

36. ตอบ b)

Temporal = ข้อมูลเดิมถูกใช้ซ้ำในเวลาใกล้กัน เช่นตัวแปร i ในทุกรอบของ loop

- a และ c เป็นตัวอย่างของ **spatial** locality (ข้อมูลที่อยู่ติดกัน)
- d เป็นการตั้งค่า cache ที่ใช้ประโยชน์จาก spatial locality

37. ตอบ b) 128

#sets = cache size / (block × ways) = 8192 / (32 × 2) = **128**

- **ตัวลวง c (256):** ลืมหารด้วย 2 ways (นั่นคือจำนวน lines)

38. ตอบ b) Dirty bit

Write-back เขียนลง memory ตอน replace เท่านั้น จึงต้องมี dirty bit บอกว่า block นี้ถูกแก้ไขแล้ว ถ้า dirty = 0 ตอน replace ไม่ต้องเขียนกลับ

39. ตอบ c) OS

Page fault จัดการด้วย **software (OS)** เพราะ hardware ไม่รู้จัก file system หรือ swap ส่วน TLB miss/cache miss เป็นงานของ hardware

40. ตอบ c) NFS

NAS ใช้ protocol ระดับไฟล์: **NFS, CIFS/SMB**, AFP, FTP

- **ตัวลวง a:** iSCSI เป็นของ **SAN** (block)
- **ตัวลวง b, d:** Fibre Channel และ SATA เป็น interface ระดับ block

Part 2: ตอบสั้น

S1. 3 ตัว: A + B = (A' · B')' = NAND(NAND(A,A), NAND(B,B))

1. NAND ตัวที่ 1: NAND(A, A) = A'
2. NAND ตัวที่ 2: NAND(B, B) = B'
3. NAND ตัวที่ 3: NAND(A', B') = (A'B')' = **A + B** (DeMorgan)

เกณฑ์: จำนวนถูก 1 คะแนน และนิพจน์ถูก 1 คะแนน

S2. F = B'D' + BD (= B ⊙ D หรือ (B ⊕ D)')

- m0, 2, 8, 10 = 0000, 0010, 1000, 1010 → B = 0 และ D = 0 (4 มุมของ K-map) → **B'D'**
- m5, 7, 13, 15 = 0101, 0111, 1101, 1111 → B = 1 และ D = 1 (สี่เหลี่ยมกลาง) → **BD**

เกณฑ์: แต่ละพจน์ 1 คะแนน

S3. B เร็วกว่า 1.11 เท่า

IC เท่ากัน จึงเทียบเวลาต่อคำสั่ง = CPI / f

- A: 2.0 / 2 GHz = **1.0 ns**
- B: 2.7 / 3 GHz = **0.9 ns**
- → B เร็วกว่า 1.0/0.9 = **1.11 เท่า**

บทเรียน: clock สูงขึ้น 1.5 เท่า แต่ CPI แย่ลง 1.35 เท่า จึงได้ประโยชน์แค่ 1.11 เท่า

S4. 1.47

1 / ((1 − 0.4) + 0.4/5) = 1 / (0.6 + 0.08) = 1/0.68 ≈ **1.47**

S5. (ก) 0x2018 (ข) 0x2180

- (ก) Vector table: Base + n × address size = 0x2000 + 6 × 4 = 0x2000 + 24 = 0x2000 + **0x18**
- (ข) Handler table: Base + n × handler size = 0x2000 + 6 × 64 = 0x2000 + 384 = 0x2000 + **0x180**

ระวัง: ต้องแปลงผลคูณจากฐานสิบเป็นฐานสิบหกก่อนบวก

S6. ADD = 190 ns, clock = 230 ns

- ADD ใช้ PC 10 + Inst MEM 40 + Control 40 + Register 20 + MUX 10 + ALU 60 + MUX 10 = **190 ns** (**ไม่ผ่าน data memory**)
- แต่ single cycle ต้องตั้ง clock ตาม**คำสั่งที่ช้าที่สุด (LW = 230 ns)**
- → ADD เสียเวลาเปล่า 40 ns ต่อคำสั่ง ซึ่งเป็นแรงจูงใจของ multi-cycle

S7. 224 cycles = 448 ns

- ideal = k + (n – 1) = 5 + 199 = 204 cycles
- load-use = 10% × 200 = 20 ครั้ง ครั้งละ stall 1 cycle → +20
- รวม 224 cycles × 2 ns = **448 ns** (CPI ที่มีผล ≈ 1.12)

S8. index = 35, tag = 1 (offset = 4)

- offset = $\log_2 16$ = 4 bits
- #lines = 4096/16 = 256 → index = 8 bits
- tag = 32 – 8 – 4 = 20 bits

0x1234 = 0001 **0010 0011** 0100:

- offset (4 บิตล่าง) = 0100 = 4
- index (8 บิตถัดไป) = 0010 0011 = 0x23 = **35**
- tag = 0001 = **1**

ตรวจด้วยฐานสิบ: 4660 mod 4096 = 564 → 564 div 16 = 35 ✓ และ 4660 div 4096 = 1 ✓

S9. CPI = 2.8

CPI = 1.2 + (1 × 0.01 × 80) + (0.25 × 0.04 × 80) = 1.2 + 0.8 + 0.8 = **2.8**

ทุกคำสั่งต้อง fetch (1 access) แต่มีเพียง 25% ที่มี data access

S10. VPN = 18 bits, page table = 1 MB

page 16 KB = 2^{14} → offset 14 bits → VPN = 32 – 14 = **18 bits** → 2^{18} × 4 B = 2^{20} B = **1 MB**

ข้อสังเกต: page ใหญ่ขึ้น page table จะเล็กลง (เทียบกับ 4 MB เมื่อ page = 4 KB)

เฉลยละเอียด ชุดที่ 4

ตารางคำตอบ (ตรวจเร็ว)

ข้อ	1	2	3	4	5	6	7	8	9	10
ตอบ	a	b	a	b	d	b	a	b	c	c
ข้อ	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**
ตอบ	b	b	c	c	b	c	a	a	a	d
ข้อ	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**
ตอบ	a	d	b	c	d	a	c	b	d	c
ข้อ	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**	**40**
ตอบ	a	b	c	a	d	b	a	b	a	c

แยกคะแนน: Digital Logic = ข้อ 1–11 + S1–S2 (เต็ม 15) · Computer Architecture = ข้อ 12–40 + S3–S10 (เต็ม 45)

ข้อไหนอยู่บทไหน:

บท	ข้อ
Digital Logic	1–11, S1, S2
Ch1 Introduction	12
Ch2 Performance	14–19, S3, S4
Ch3 ISA	20–25, S5
Ch4 Single cycle	26, 27
Ch5 Multi cycle	28, 29, S6
Ch6 Pipelining	13, 30–33, S7
Ch7 ILP	34, 35
Ch8 Memory	36–39, S8, S9
Ch9 Storage	40, S10

Part 1: Choice

Digital Logic

> **1. ตอบ a) 0xD6**
>
> แบ่งทีละ 4 บิต: 1101 = D และ 0110 = 6 → **0xD6** (ตัวลวง b สลับลำดับ nibble)

2. ตอบ b) 229

3 × 64 + 4 × 8 + 5 = 192 + 32 + 5 = **229**

3. ตอบ a) 0 ถึง 1023

Unsigned n bits: 0 ถึง $2^n - 1 = 2^{10} - 1 = 1023$

- **ตัวลวง c:** ลืมลบ 1
- **ตัวลวง b:** เป็นช่วงของ 2's complement

4. ตอบ b) 1010 0101

+90 = 0101 1010 → 1's complement = **กลับทุกบิต** = 1010 0101

- **ตัวลวง a (1010 0110):** เป็น **2's complement** (กลับบิตแล้วบวก 1)
- **ตัวลวง d:** เป็น +90

5. ตอบ d) carry out = 1, overflow = 1

1001 0000 + 1010 0000 = **1** 0011 0000

- **carry out = 1**
- ผลคือ 0x30 = +48 ซึ่งเป็น **ลบ + ลบ ได้บวก** → **overflow = 1** ((−112) + (−96) = −208 ต่ำกว่า −128)

6. ตอบ b) A'B + AB'

XOR เป็น 1 เมื่ออินพุต**ต่างกัน** → แถว 01 (A'B) และ 10 (AB')

- **ตัวลวง a:** เป็น **XNOR** (เท่ากัน)

7. ตอบ a) AB + A'C

Consensus theorem: พจน์ BC เป็น consensus ของ AB กับ A'C จึงตัดทิ้งได้

ตรวจ: ถ้า B = C = 1 แล้ว A = 1 ทำให้ AB = 1 และ A = 0 ทำให้ A'C = 1 → BC ถูกครอบคลุมเสมอ

8. ตอบ b) C'

m0, 2, 4, 6 = 000, 010, 100, 110 → **C = 0 ทุกตัว** → F = C'

9. ตอบ c) 7

ต่อเป็นต้นไม้ 3 ชั้น: 4 + 2 + 1 = **7** ตัว (8 อินพุตลดลงทีละครึ่งจนเหลือ 1)

- **ตัวลวง a (3):** เป็นจำนวน select lines

10. ตอบ c) D flip-flop

Flip-flop = edge-triggered ส่วน **latch = level-sensitive** (โปร่งใสตลอดช่วงที่ enable) และ combinational circuit ไม่มีหน่วยความจำเลย

11. ตอบ b) 3

Mealy ไม่ต้องมี state "เห็นครบ 101" เพราะ output อยู่บนเส้น transition

- S0 = เริ่มต้น, S1 = เห็น "1", S2 = เห็น "10"
- จาก S2 ได้ 1 → output 1 แล้วไป S1 (overlapping)

(Moore ต้องใช้ 4 states ซึ่งเป็นตัวลวง c)

Computer Architecture

12. ตอบ b) ISA

ลำดับชั้นในสไลด์: Application → OS/Compiler → **ISA** → Firmware → Processor/I/O → Datapath & Control → Digital design → Circuit → Layout

ISA คือ interface ระหว่าง software กับ hardware

13. ตอบ c) Throughput

Lessons learned: "Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload"

14. ตอบ c) Elapsed (wall-clock) time

- **real** = เวลาจริงที่ผ่านไป
- **user** = CPU time ใน user mode
- **sys** = CPU time ใน kernel

15. ตอบ b) 1.5

$T_X / T_Y = 30/20 =$ **1.5**

- **ตัวลวง a (0.67):** เป็น "X เร็วกว่า Y กี่เท่า" (คือช้ากว่า)

16. ตอบ c) 2 s

CPU time = cycles / f = $3 \times 10^9 / 1.5 \times 10^9 =$ **2 s**

- **ตัวลวง d (4.5 s):** คูณแทนหาร

17. ตอบ a) 1.7

0.6(1) + 0.3(2) + 0.1(5) = 0.6 + 0.6 + 0.5 = **1.7**

- **ตัวลวง b (2.67):** เฉลี่ย 1, 2, 5 แบบไม่ถ่วงน้ำหนัก

18. ตอบ a) 1.33

1 / (0.5 + 0.5/2) = 1/0.75 = **1.33**

- **ตัวลวง c (2):** คิดว่าทั้งโปรแกรมเร็วขึ้นเท่าส่วนที่เร่ง

19. ตอบ a) 4

GM = √(2 × 8) = √16 = **4**

- **ตัวลวง b (5):** เป็น arithmetic mean
- **ตัวลวง d (3.2):** เป็น harmonic mean

20. ตอบ d)

Accumulator มี register เดียว ค่ากลางจึงต้องเก็บลง memory บ่อย (memory traffic สูง) และ ACC เป็น bottleneck ไม่เหมาะกับ pipeline

- **ตัวลวง a:** กลับกัน คือ accumulator ใช้ hardware น้อยที่สุด

21. ตอบ a)

สไลด์: "Intel ISA is considered CISC, but it is translated into internal RISC code before performing parallelism" เส้นแบ่งระหว่าง CISC กับ RISC จึงไม่ชัดอีกต่อไป

22. ตอบ d) 8

double ขนาด 8 bytes ต้องเริ่มที่ address mod 8 = 0 → a อยู่ที่ 0, padding 1–7, **b อยู่ที่ 8–15**

23. ตอบ b) Strength reduction

แทน operation ที่แพง (คูณ) ด้วย operation ที่ถูกกว่า (shift) เป็น machine-dependent optimization

24. ตอบ c) Relocatable

ปลายทาง = PC + displacement ไม่ผูกกับ address จริง ย้ายโปรแกรมไปวางที่ไหนก็ยังถูก และ branch ส่วนใหญ่กระโดดใกล้ ๆ จึงใช้ field เล็กได้

- **ตัวลวง a:** PC-relative กระโดดได้แค่ในระยะของ field ไปทุก address ไม่ได้

25. ตอบ d)

Callee-save: subroutine PUSH เฉพาะ register ที่จะใช้แล้ว POP ก่อน RET → ไม่ save เกินจำเป็น (สไลด์ถามว่า "Callee save has more benefit?")

- **ตัวลวง a:** เป็นแบบ caller-save หรือ PUSHA

26. ตอบ a)

BEQ: alu_ops = 10 (SUB เพื่อดู zero), sel_addpc = zero, sel_b = 0 (เทียบกับ R[rt]), reg_wr = 0, mem_wr = 0

- **ตัวลวง b:** เป็นค่าของ ADD
- **ตัวลวง d:** เป็นค่าของ LW

27. ตอบ c) อ่าน 2, เขียน 1

อ่าน R[rs] และ R[rt] พร้อมกันได้ แต่เขียนได้ทีละตัว ซึ่งเป็นต้นเหตุของ structural hazard เมื่อ 2 คำสั่ง WB พร้อมกันใน pipeline

28. ตอบ b) 825 ns

ADD 4 × 4 = 16, SW 2 × 4 = 8, BEQ 2 × 3 = 6, JMP 1 × 3 = 3 → **33 cycles** × 25 = **825 ns**

- **ตัวลวง a:** นับแบบ single cycle (9 × 25)
- **ตัวลวง d:** คิดว่าทุกคำสั่งใช้ 5 cycles

29. ตอบ d)

Microinstruction เป็นส่วนของ **organization** เปลี่ยนรุ่น CPU แล้วโปรแกรมใช้ไม่ได้ ส่วน ISA (macroinstruction) เป็น abstraction ที่ทำให้ software portable

30. ตอบ c)

แก้ structural hazard ด้วยการ**เพิ่ม resource** ("make two copies") ให้มี I-cache สำหรับ IF และ D-cache สำหรับ MEM ทำงานพร้อมกันได้

31. ตอบ a) RAW

b อ่าน a ซึ่ง**เพิ่งถูกเขียน** → Read After Write (ตัวอย่างตรงจากสไลด์) และแก้ได้ด้วย forwarding

32. ตอบ b) 1

Load-use: ข้อมูลพร้อมตอนจบ MEM แต่คำสั่งถัดไปต้องใช้ตอนต้น EX ใน cycle เดียวกัน forward ย้อนเวลาไม่ได้ จึงต้อง stall 1 cycle

- **ตัวลวง d (3):** เป็นกรณีที่ไม่มี forwarding เลย

33. ตอบ c) 90%

สไลด์: ทายแบบคงที่ได้อย่างน้อย 50% ส่วน "more advance scheme 90% correct (e.g. branch history)"

34. ตอบ a) 254

$T = (k + (n - x)/x)\ T_c = 5 + 996/4 = 5 + 249 =$ **254**

- **ตัวลวง b (1004):** เป็น pipeline ธรรมดา
- **ตัวลวง c (250):** ลืมเวลา fill

35. ตอบ d)

Dynamic scheduling จัดลำดับตอนรันด้วย hardware binary เดิมจึงได้ประโยชน์ทันที

- **ตัวลวง a, b, c:** เป็นข้อดีของ **static** scheduling ทั้งหมด

36. ตอบ b) Capacity

สไลด์เรียก ping-pong effect ว่า "extreme capacity miss" (cache เล็กเกินกว่าจะเก็บ working set ได้)

37. ตอบ a)

block ใหญ่เกินไปที่ cache size คงที่จะทำให้:

- จำนวน block น้อยลง → **temporal locality แย่ลง miss rate กลับเพิ่ม**
- block ใหญ่ขึ้น → **miss penalty สูงขึ้น** (ต้องย้ายข้อมูลมากขึ้นผ่าน bus เดิม)

ตัวลวง d: compulsory miss ยังมีอยู่เสมอ แม้ block ใหญ่จะช่วยลดลงได้บ้าง

38. ตอบ b) 4-way

เงื่อนไข: cache size / ways ≤ page size → 16 KB / ways ≤ 4 KB → **ways ≥ 4**

39. ตอบ a)

Miss penalty ของดิสก์สูงมาก ถ้า write-through ทุกการเขียนต้องไปถึงดิสก์ จึงใช้ write-back (page out ตอนถูก replace) เท่านั้น

40. ตอบ c) 16 TB

RAID 6 เสียพื้นที่ให้ parity 2 ลูก: (6 – 2) × 4 = **16 TB** และทนดิสก์เสียได้ 2 ลูก

- **ตัวลวง b (20 TB):** คิดแบบ RAID 5
- **ตัวลวง a (24 TB):** คิดแบบ RAID 0

Part 2: ตอบสั้น

S1. (ก) J = 1, K = X (ข) T = 1

- JK: Q 0 → 1 ทำได้ทั้ง set (J = 1, K = 0) และ toggle (J = 1, K = 1) ดังนั้น **K เป็น don't care**
- T: ต้องการให้ค่าเปลี่ยน จึงใช้ **T = 1**

เกณฑ์: ข้อละ 1 คะแนน

S2. 1101 1011 = 0xDB

1. +37 = 0010 0101
2. กลับบิตได้ 1101 1010
3. บวก 1 ได้ **1101 1011 = 0xDB**

ตรวจ: 0xDB = 219 และ 219 – 256 = –37 ✓

S3. CPU time = 3 s, MIPS = 2000

- T = $6 \times 10^9 \times 1.25 / 2.5 \times 10^9 = 7.5/2.5 =$ **3 s**
- MIPS = IC / (T × 10^6) = $6 \times 10^9 / 3 \times 10^6 =$ **2000** (ตรวจ: f / (CPI × 10^6) = 2500/1.25 = 2000 ✓)

S4. A = 11.6 s, B = 10.2 s → เลือก B

- A: 0.7 × 8 + 0.3 × 20 = 5.6 + 6.0 = **11.6**
- B: 0.7 × 12 + 0.3 × 6 = 8.4 + 1.8 = **10.2**

บทเรียน: A เร็วกว่าในโปรแกรมที่ใช้บ่อย (P1) แต่ช้ามากใน P2 จนเฉลี่ยแล้วแพ้ → "เร็วที่สุด" ขึ้นกับ workload

S5. little: 0D 0C 0B 0A, big ที่ 0x102 = 0x0C

Address	0x100	0x101	0x102	0x103
Little endian	**0D**	0C	0B	0A
Big endian	0A	0B	**0C**	0D

S6. CPI = 4.05, multi-cycle เร็วกว่าเล็กน้อย

1. CPI = 0.25(5) + 0.10(4) + 0.45(4) + 0.15(3) + 0.05(3) = 1.25 + 0.40 + 1.80 + 0.45 + 0.15 = **4.05**
2. เวลาต่อคำสั่ง: multi = 4.05 × 60 = **243 ns** และ single = 250 ns
3. → multi-cycle เร็วกว่า 250/243 ≈ **1.03 เท่า** (ต่างกันน้อยมาก)

เกณฑ์ตัดสิน: multi-cycle ชนะเมื่อ CPI < 250/60 ≈ 4.17

S7. Dependencies + เหตุผลที่ย้าย SUB ไม่ได้

คู่	Register	ชนิด
1 → 2	r2	**RAW** (load-use → stall 1)
2 → 4	r3	**RAW** (forward ได้)
1 → 3	r2	**WAW**
2 → 3	r2	**WAR**

- **ทำไมย้ายไม่ได้:** ถ้าย้าย SUB ไปไว้ก่อน ADD จะทำให้ ADD อ่าน r2 ที่ SUB เขียน แทนค่าที่ LW โหลด (**ละเมิด WAR**) และค่าสุดท้ายของ r2 จะกลายเป็นค่าจาก LW แทน SUB (**ละเมิด WAW**) ผลลัพธ์จึงผิด
- **วิธีแก้: register renaming** โดยให้ SUB เขียนลง register อื่น (เช่น r7) และแก้คำสั่งหลังจากนั้นที่ใช้ r2 ให้ใช้ r7 แทน จากนั้นจึงย้าย SUB มาคั่นระหว่าง LW กับ ADD เพื่อลบ stall ได้

เกณฑ์: ระบุ dependency ครบ 1 คะแนน และอธิบายเหตุผลพร้อมวิธีแก้ 1 คะแนน

S8. tag 18, index 9, offset 5, ใช้ 2 comparators

1. offset = $\log_2 32 = 5$
2. #sets = 32768 / (32 × 2) = 512 → index = 9
3. tag = 32 − 9 − 5 = 18
4. ใช้ 2 comparators (หนึ่งตัวต่อ way)

S9. แบบเดิม 6 cycles, 2-way 5.5 cycles → 2-way ดีกว่า

- แบบเดิม: 2 + 0.08 × 50 = 2 + 4 = **6 cycles**
- 2-way: 2.5 + 0.06 × 50 = 2.5 + 3 = **5.5 cycles**

บทเรียน: associativity แลก hit time ที่เพิ่มขึ้นกับ miss rate ที่ลดลง ต้องคำนวณ AMAT จึงจะตัดสินได้

S10. RAID กับดิสก์ 3 TB × 5

Level	พื้นที่ใช้งาน	เสียได้
RAID 0	5 × 3 = **15 TB**	**0 ลูก**
RAID 5	(5 – 1) × 3 = **12 TB**	**1 ลูก**
RAID 6	(5 – 2) × 3 = **9 TB**	**2 ลูก**

เกณฑ์: ถูกครบ 3 แถวได้ 2 คะแนน ถูก 2 แถวได้ 1 คะแนน

แปลผลชุดที่ 3–4 (เต็มชุดละ 60)

คะแนน	ระดับ	ทำต่อ
51–60	พร้อมมาก	อ่านกับดักรวมใน Exam Notes ก่อนสอบ
42–50	พร้อม	ใช้ตาราง "ข้อไหนอยู่บทไหน" หาบทที่ผิด แล้วอ่านเฉพาะส่วนนั้น
30–41	ปานกลาง	อ่าน Exam Notes บทที่ผิดใหม่ แล้วทำชุดที่ทำผิดเยอะซ้ำ
< 30	ต้องเร่ง	เริ่มจาก Formula Sheet แล้วไล่ตัวอย่างในแต่ละบท